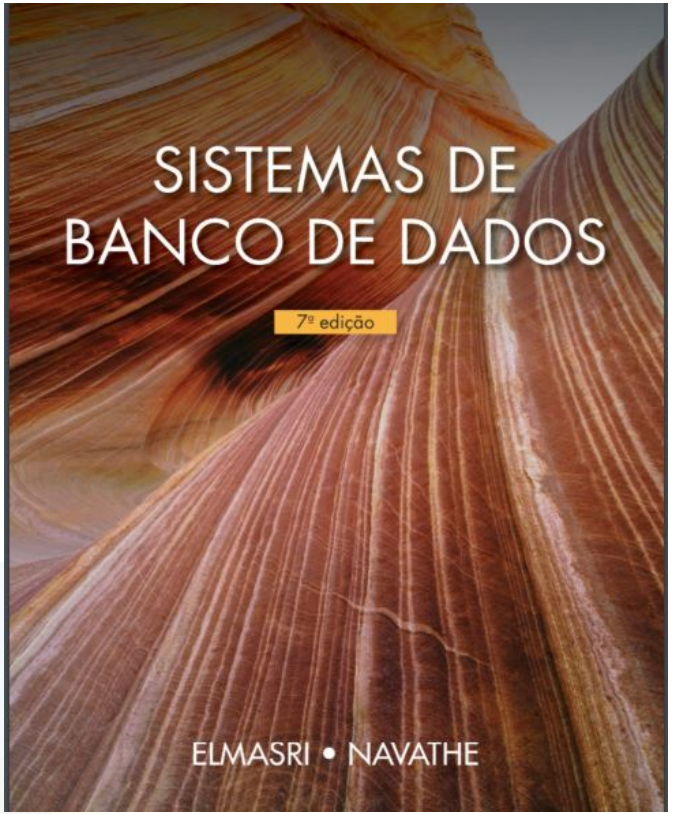
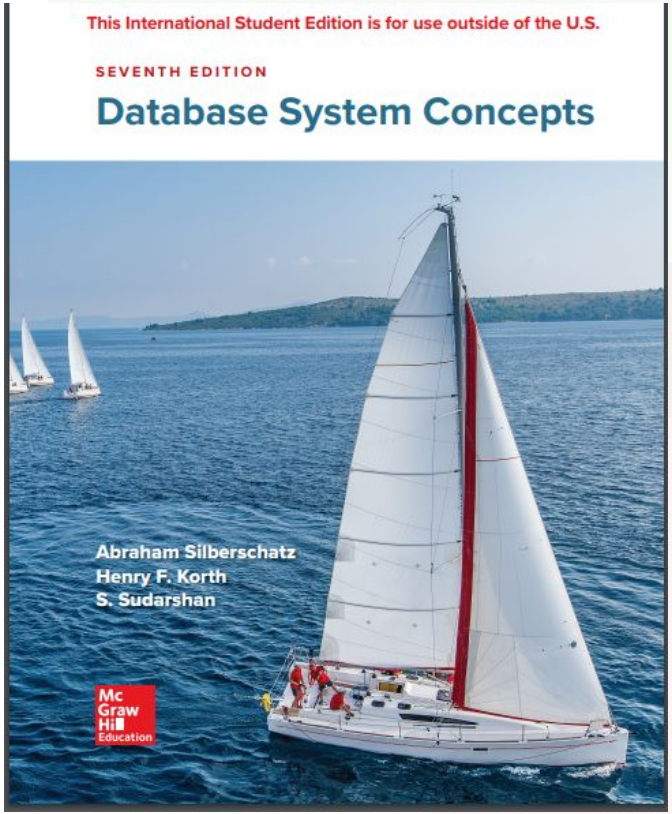
Processamento de Transações

Vocês confiariam em um banco que perde dinheiro quando falta energia?

robespierre.pita@ufba.br

Referências





.pita@ufba.br

POR QUE TRANSAÇÕES EXISTEM?

Transações garantem que um conjunto de operações aconteça por inteiro ou não aconteça.

SISTEMA	OPERAÇÕES DA TRANSAÇÃO (EXEMPLOS)	O QUE ACONTECERIA SEM TRANSAÇÃO?	CONSEQUÊNCIA
BANCO Transferência	1. Debitar conta A 2. Creditar conta B A → R$ 50 → B	Sistema debita da conta A... ... mas falha antes de creditar a conta B.	DINHEIRO DESAPARECE!
iFood Criação de pedido	1. Criar pedido 2. Baixar estoque 3. Cobrar cliente	Cliente é cobrado... ... mas o pedido não é criado ou o estoque não é baixado.	COBRANÇA SEM PEDIDO OU ESTOQUE INCORRETO!
Uber Cobrança	1. Finalizar corrida 2. Cobrar cliente 3. Pagar motorista	Falha no sistema pode cobrar duas vezes ou não pagar o motorista.	COBRANÇA DUPLICADA OU CORRIDA GRÁTIS!
HOSPITAL Alta médica	1. Registrar alta do paciente 2. Liberar leito 3. Atualizar prontuário e faturamento	Leito é liberado, mas alta não é registrada, ou faturamento não é gerado.	LEITO OCUPADO INCORRETAMENTE E ERRO OPERACIONAL!

O QUE PODE DAR ERRADO SEM TRANSAÇÕES?

1. Dinheiro sumindo



Falha antes de creditar conta B.

R$ 50 perdidos!

2. Estoque negativo



Dois pedidos confirmados para o último item.

Estoque negativo!

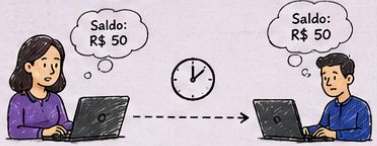
3. Cobrança duplicada



Falha de comunicação faz a cobrança acontecer duas vezes.

Cliente cobrado em dobro!

4. Dados inconsistentes



Duas pessoas acessam ao mesmo tempo e veem informações diferentes.

Decisões erradas!

Transações existem para garantir Atomicidade, Consistência, Isolamento e Durabilidade (ACID) e manter nossos dados corretos, mesmo com múltiplos usuários e falhas.

Imagem aprimorada usando o ChatGPT

robespierre.pita@ufba.br

Conceito de Transação

Uma Transação é uma unidade de execução de programa (abstração?) que acessa e possivelmente atualiza vários itens de dados.

Duas questões para lidar: 

- Diversas falhas podem acontecer durante a execução de um programa.
- Múltiplos programas podem executar de forma concorrente.

Exemplo de transação para transferir R$ 50 de conta A para conta B:

```
1.  read(A)
2.  A := A - 50
3.  write(A)
4.  read(B)
5.  B := B + 50
6.  write(B)
```

robespierre.pita@ufba.br

Propriedades requeridas para uma transação

Atomicidade

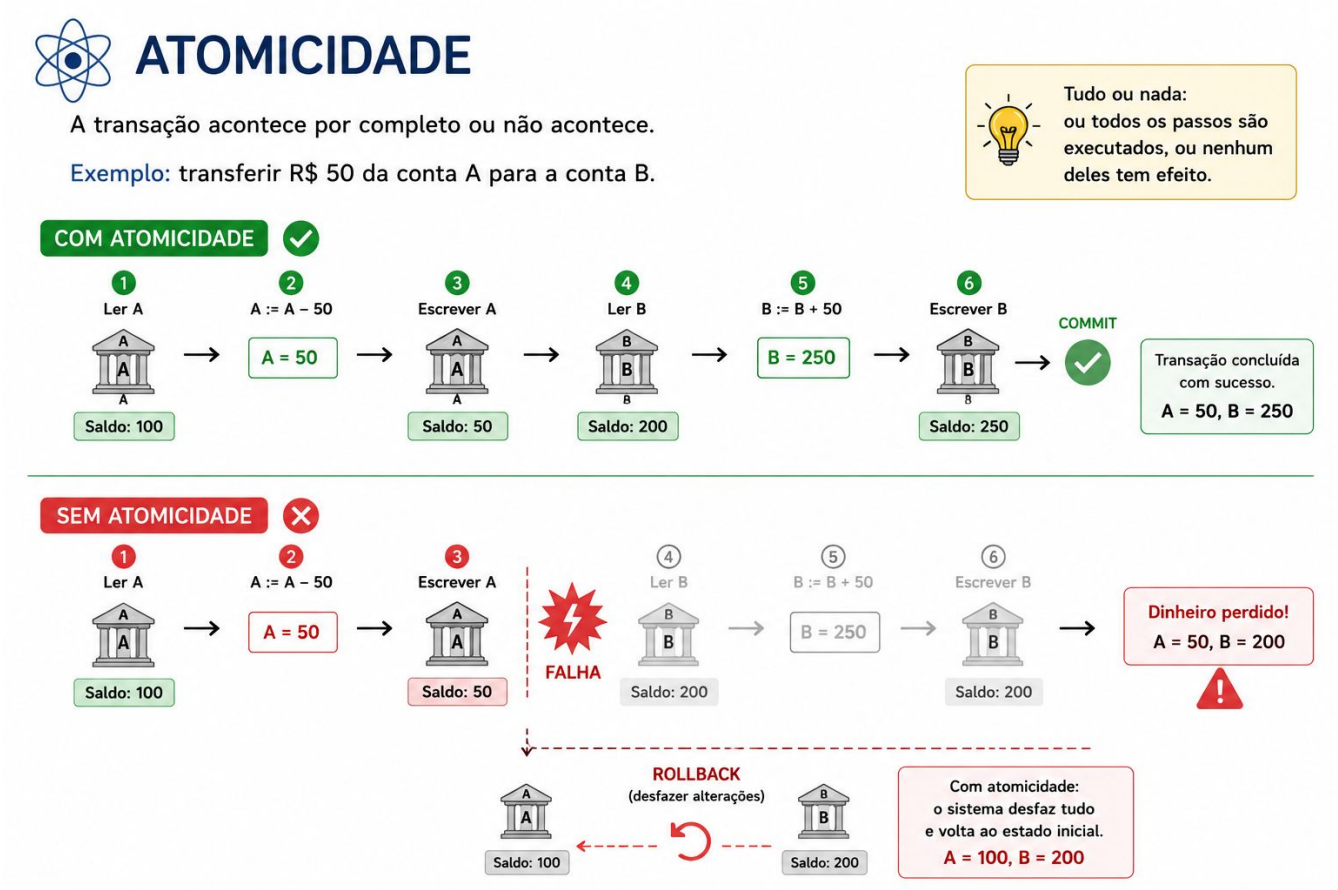
- Se a transação falha após o passo 3 e antes do passo 6, o dinheiro estará “perdido”, levando a um estado inconsistente da base de dados.
- O sistema pode garantir que atualizações de uma transação parcialmente executada não sejam refletidas no banco de dados.

Exemplo de transação para transferir R$ 50 de conta A para conta B:

```
1.  read(A)
2.  A := A - 50
3.  write(A)
4.  read(B)
5.  B := B + 50
6.  write(B)
```

robespierre.pita@ufba.br

Propriedades requeridas para uma transação



Propriedades requeridas para uma transação

Durabilidade

- Uma vez que a transação foi completada, e o usuário recebeu a notificação de sucesso da operação), as atualizações devem persistir no banco mesmo em caso de falhas no software ou hardware.

Exemplo de transação para transferir R$ 50 de conta A para conta B:

```
1.  read(A)
2.  A := A - 50
3.  write(A)
4.  read(B)
5.  B := B + 50
6.  write(B)
```

robespierre.pita@ufba.br

Propriedades requeridas para uma transação

DURABILIDADE

Uma vez que a transação foi completada (commit) e o usuário recebe a confirmação de sucesso, **as atualizações persistem no banco** mesmo em caso de falhas de software ou hardware.

Durabilidade = depois do commit, os dados são definitivos.

FLUXO NORMAL

1. Transação executa

```
BEGIN;
UPDATE contas
SET saldo = saldo - 50
WHERE id = 1;
...
```

2. Commit

COMMIT

Usuário recebe confirmação

3. Registro no log (WAL)

LOG

A operação é registrada no log de transações antes de ser confirmada.

4. Gravação no disco

Os dados são gravados de forma permanente no disco.

5. Durabilidade garantida

Mesmo que algo aconteça depois, os dados estarão seguros no disco.

E SE HOUVER UMA FALHA?

1. Após o commit, ocorre uma falha

Servidor cai, energia acaba, etc.

2. Sistema reinicia

O banco é reiniciado e verifica o log (WAL).

3. Log mostra que o commit foi concluído

LOG

Como o commit já estava registrado no log, a transação é considerada permanente.

4. Dados permanecem no disco

Os dados continuam lá!

A = 50

B = 250

Durabilidade garante que, uma vez confirmada, a transação sobrevive a falhas.
É como um contrato: depois do "ok", não há volta.

write-ahead logging

WAL é uma técnica em que se registra o log da operação antes da modificação real dos dados.

O QUE FICA NO WAL?

O WAL registra as mudanças de forma sequencial e imutável.

Exemplo de registro no log

TXN_ID	TIPO	TABELA	DADOS ANTES	DADOS DEPOIS
125	UPDATE	contas	saldo=100	saldo=50

Esse registro permite REFAZER (redo) ou DESFAZER (undo) a operação se necessário.

Já ouviu falar em **CDC (*Change Data Capture*)**?

robespierre.pita@ufba.br

Propriedades requeridas para uma transação

Consistência

- A soma entre A e B permanece a mesma depois da execução da transação.
- A consistência pode ser implícita ou explícita:
  - implícita: Operações aritméticas devem ter o mesmo resultado.
  - explícita: restrições de integridade (pks e fks) são preservadas.

- Uma base pode estar temporariamente inconsistente durante a execução.

Exemplo de transação para transferir R$ 50 de conta A para conta B:

```
1. read(A)
2. A := A - 50
3. write(A)
4. read(B)
5. B := B + 50
6. write(B)
```



robespierre.pita@ufba.br

Propriedades requeridas para uma transação

Isolamento

- Caso uma segunda transação consulte a base de dados entre os passos 3 e 6, ela encontrará valores inconsistentes (os valores de A + B serão menores do que deveriam).

- O isolamento pode ser alcançado rodando transações serialmente.



robespierre.pita@ufba.br

Propriedades ACID

Atomicidade: As operações somente se refletem na base de dados se executaram com sucesso.
Consistência: Transações preservam a consistência da base.
Isolamento: As transações vão executar sem o conhecimento de operações concorrentes. Resultados intermediários das transações devem ser escondidos de transações concorrentes.

- Ou seja, para cada par de transações T_i e T_j, aparece para T_i que i) T_i terminou sua execução antes que T_j tenha iniciado ou que ii) T_j iniciou depois de T_i.

Durabilidade: Após uma execução com sucesso, as mudanças feitas são persistidas mesmo com a presença de falhas.

robespierre.pita@ufba.br

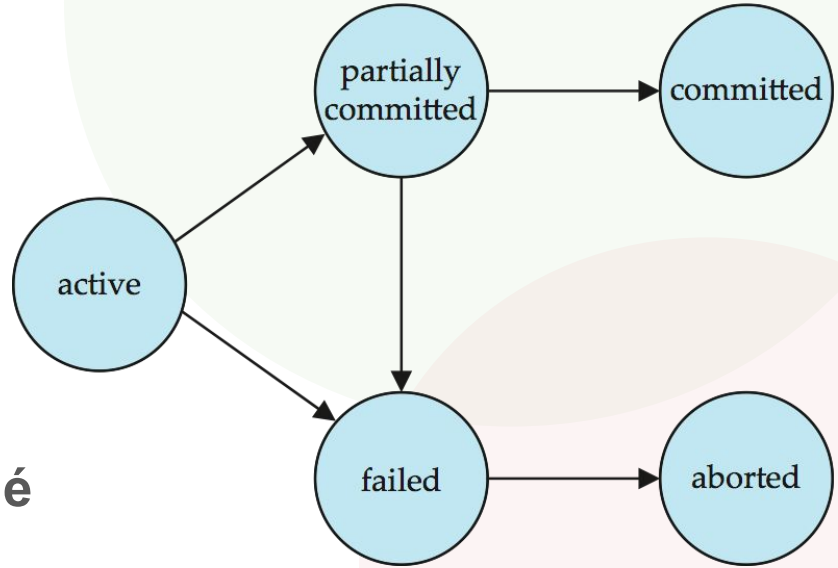
Execuções concorrentes

As vantagens de executar múltiplas transações:

- Uso ótimo de recursos (processador e discos), garantindo maior vazão (*throughput*).
- Diminuição do tempo de resposta (latência).

Nestes casos, para garantir isolamento, é preciso usar esquemas de controle de concorrência.

Estados de transação



robespierre.pita@ufba.br

Escalonamento

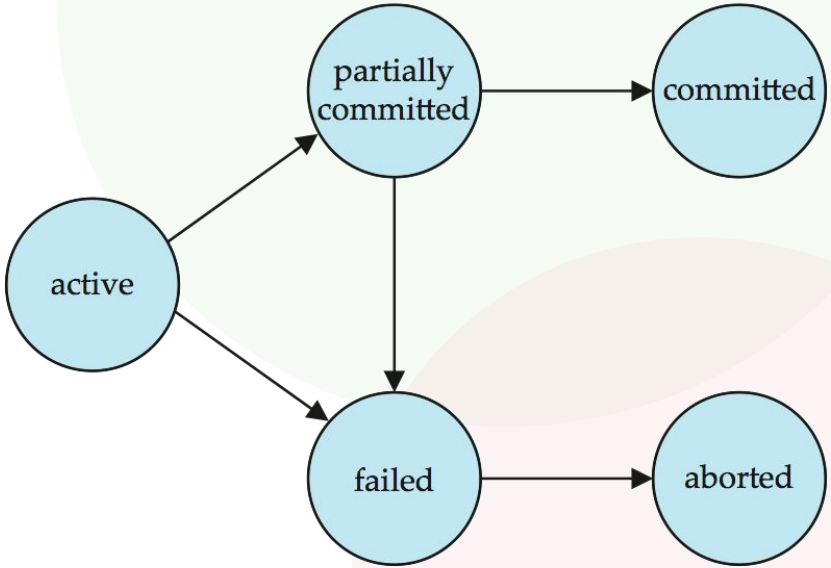
Um escalonamento define uma ordem cronológica de execução das instruções/transações concorrentes.

- Deve conter todas as instruções de cada transação.
- Deve preservar a ordem interna das instruções em cada transação.

COMMIT: Instrução final de uma transação executada com sucesso.

ABORT: instrução final de uma transação que falha ou é cancelada.

Estados de transação



robespierre.pita@ufba.br

Escalonamentos 1 e 2

Considere um cenário em que T1 transfere R$ 50 de A para B e T2 transfere 10% do saldo de A para B.

Escalonamento serial:

T_1	T_2
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit

T_1	T_2
	read (A) $temp := A * 0.1$ $A := A - temp$ write (A) read (B) $B := B + temp$ write (B) commit
read (A) $A := A - 50$ write (A) read (B) $B := B + 50$ write (B) commit	

robespierre.pita@ufba.br

Escalonamento 3

Considere um cenário em que T1 transfere R$ 50 de A para B e T2 transfere 10% do saldo de A para B.

O escalonamento a seguir não é serial, mas equivalente:

T_1	T_2
read (A)	
$A := A - 50$	
write (A)	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
read (B)	
$B := B + 50$	
write (B)	
commit	
	read (B)
	$B := B + temp$
	write (B)
	commit

robespierre.pita@ufba.br

Escalonamento 4

Considere um cenário em que T1 transfere R$ 50 de A para B e T2 transfere 10% do saldo de A para B.

O escalonamento a seguir não preserva a soma A + B:

T_1	T_2
read (A)	
$A := A - 50$	
	read (A)
	$temp := A * 0.1$
	$A := A - temp$
	write (A)
	read (B)
write (A)	
read (B)	
$B := B + 50$	
write (B)	
commit	
	$B := B + temp$
	write (B)
	commit

robespierre.pita@ufba.br

Transações serializáveis

Suposição básica: Cada transição preserva a consistência da base. Portanto, a execução serial das transações também preserva a consistência da base.

Desta forma: um escalonamento é serializável SSE seu resultado é equivalente a um escalonamento serial. Duas noções de equivalência de escalonamento surgem, a “conflict serializability” e a “view serializability”.

T_1	T_2
read (A)	
write (A)	
	read (A)
	write (A)
read (B)	
write (B)	
	read (B)
	write (B)

T_1	T_2
read (A)	
write (A)	
read (B)	
write (B)	
	read (A)
	write (A)
	read (B)
	write (B)

robespierre.pita@ufba.br

Transações serializáveis

Suposição básica: Cada transição preserva a consistência da base. Portanto, a execução serial das transações também preserva a consistência da base.

Desta forma: um escalonamento é serializável SSE seu resultado é equivalente a um escalonamento serial. Duas noções de equivalência de escalonamento surgem, a "conflict serializability" e a "view serializability".

1. I_i = **read**(Q), I_j = **read**(Q). I_i and I_j don't conflict.
2. I_i = **read**(Q), I_j = **write**(Q). They conflict.
3. I_i = **write**(Q), I_j = **read**(Q). They conflict
4. I_i = **write**(Q), I_j = **write**(Q). They conflict

T_3	T_4
read (Q)	
	write (Q)
write (Q)	

robespierre.pita@ufba.br

Transações serializáveis

Suposição básica: Cada transição preserva a consistência da base. Portanto, a execução serial das transações também preserva a consistência da base.

Desta forma: um escalonamento é serializável SSE seu resultado é equivalente a um escalonamento serial. Duas noções de equivalência de escalonamento surgem, a “conflict serializability” e a “view serializability”.

T_{27}	T_{28}	T_{29}
read (Q)		
	write (Q)	
write (Q)		
		write (Q)

Escritas cegas acontecem quando uma transação escreve um valor em um item de dados sem antes ler o valor atual deste item.

Obrigado!!

robespierre.pita@ufba.br